Computer Science & IT



Data Structure & programming

Queue

Lecture No. 02



By- Abhishek Sir

Recap of Previous Lecture



- Topic
- Topic
- Topic
- Topic
- Topic

Linear queue

front rear

Enqueue & dequeue.

Slide

Topics to be Covered



- Topic — Circular queue
- Topic — Implementation of queue using stack
- Topic — Deque, priority queue
- Topic
- Topic

Slide



Topic : Question

A function f defined on stacks of integers satisfies the following properties. $f(\phi) = 0$ and f(push (S,i)) = max (f(s), 0) +I for all stacks S and integers i. If a stack S contains the integers 2, –3, 2, –1, 2 in order from bottom to top, what is f(S)?

(A) 6 (B) 4

(C) 3 (D) 2



$f(\phi) = 0$

$f(S) = f(\text{push}(S,i))$

5 ↑ top

$f(\text{push}(S,2))$

↓ 4 element



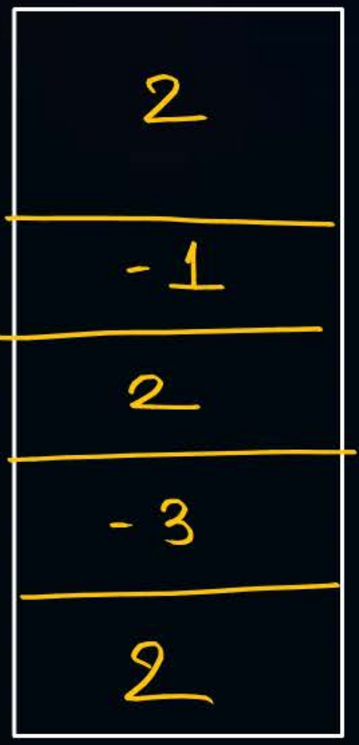


Topic : Question

A function f defined on stacks of integers satisfies the following properties. f(ϕ) = 0 and f(push (S,i)) = max (f(s), 0) +I for all stacks S and integers i. If a stack S contains the integers 2, –3, 2, –1, 2 in order from bottom to top, what is f(S)?

(A) 6 (B) 4

(C) 3 (D) 2



$f(\phi) = 0$

$f(s) = f(\text{push}(S,i))$

s ↑ 5 , top

$f(\text{push}(S,2))$

S ↓ 4

$\max(f(s),0) + i$

$= \max(f(s),0) + 2$

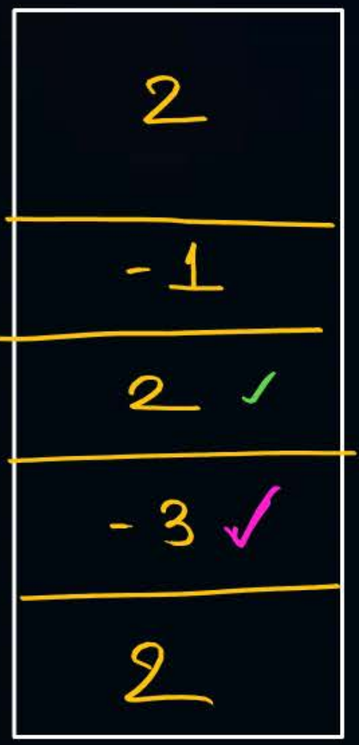


Topic : Question

A function f defined on stacks of integers satisfies the following properties. f(ϕ) = 0 and f(push (S,i)) = max (f(s), 0) +I for all stacks S and integers i. If a stack S contains the integers 2, –3, 2, –1, 2 in order from bottom to top, what is f(S)?

(A) 6 (B) 4

(C) 3 (D) 2



$f(s) = f(push(s,2))$

$= max(f(s),0) + 2$

$= 1+2$

$f(push(s,-1))$

$= max(f(s),0) - 1$

$= 2-1 = 1$

$f(s) = f(push(s,2))$

$= max(f(s),0) + 2 = 0+2$

$f(s) = f(push(s,-3))$

$= max(f(s),0) - 3 = 2-3$

$f(s) = f(push(s,2))$

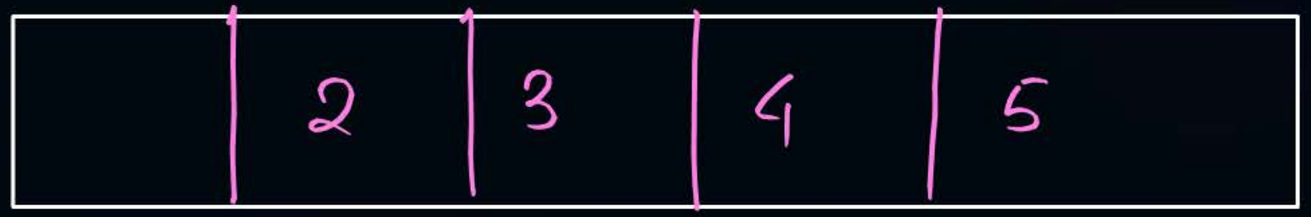
$= max(f(s),0) + 2 = 0+2$



Topic : Circular Queue

rear = 4

front = 1



define Max 100

Linear queue Enqueue (6) — overflow

Max = 5

% Remainder

rear = 4

rear = (rear + 1) % Max

= (4+1) % 5

= 5 % 5 = 0

rear = 2

rear = (rear + 1) % Max

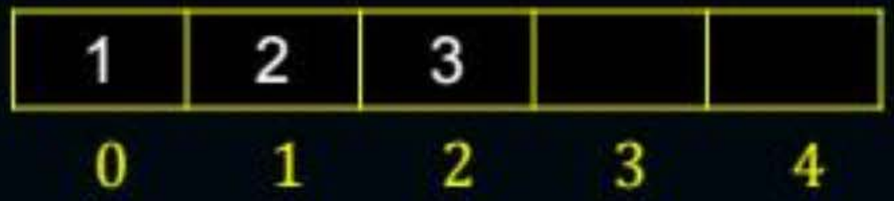
= (2+1) % 5 = 3 % 5 = 3

Slide

```
rear  = 2;
front = 0;
```



```
rear  = 2;
front = 0;
```




```
rear  = 3;
front = 0;
```

Insert 4



```
rear  = 3;
front = 0;
```

Insert 5

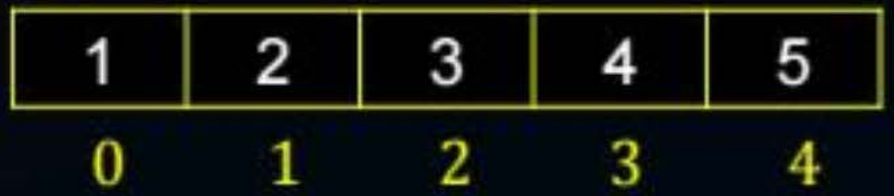


```
rear  = 4;
front = 0;
```

Insert 5




```
rear  = 4;
front = 0;
```



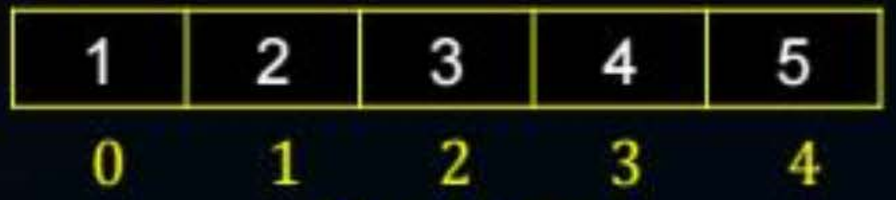
In Linear queue front <= rear

```
rear  = 4;
front = 0;
```

Insert 6




```
rear  = 4;
front = 0;
```



overflow

(rear == Max-1)

overflow

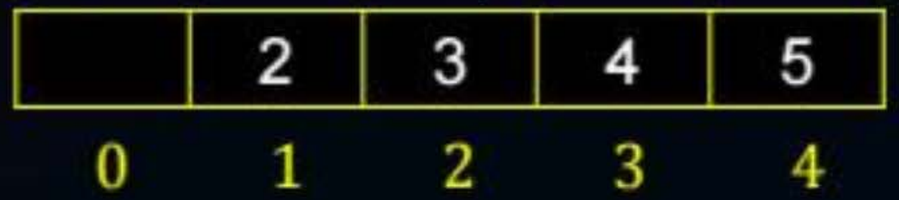
```
rear  = 4;
front = 0;
```

Delete




```
rear  = 4;
front = 1;
```

Linear queue overflow



rear = 4

front = 1



rear = 0

front = 1



Insert = overflow

Linear queue

rear = (rear + 1) % 5

= (4 + 1) % 5 = 0



Insert (6)

Condition for overflow

$$(front == 0 \;\&\&\; rear == Max - 1)$$

↑ first condition

```
rear  = 4;
front = 1;
```

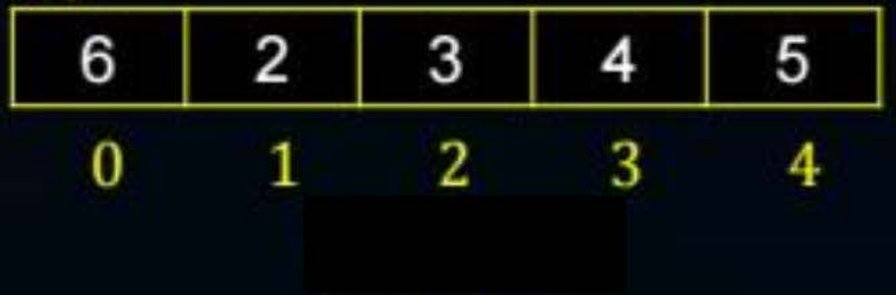
Insert 6



Overflow

```
rear  = 0;
front = 1;
```

Insert 6



```
rear  = 0;
front = 1;
```

Delete




```
rear  = 0;
front = 2;
```



~~Overflow~~

↓

Insert (7)

overflow

front == rear + 1

rear < front Circular queue

```
rear  = 0;
front = 2;
```

Insert (7)




```
rear  = 1;
front = 2;
```



rear=1

front=2

Insert(8)



overflow

Condition I

(front == 0 && rear == Max-1)

Condition II

(front == rear+1)



Topic : Circular Queue

Condition I front == 0 && rear = Max-1

0 == (4+1)%5

0 == 0

front = (rear+1)%Max

Condition II : front == rear+1

front 2

rear = 1

front == (rear+1)%Max

2 == (1+1)%5

2 == 2

Slide



Topic : Circular Queue

```
void Enqueue(int data){
    if(front==(rear+1)%Max){
        print("Queue is full");
        return;
    }
    rear=(rear+1)%Max;
    a[rear]=data;
    if(front==-1)
        front=0;
}
```

Slide



Topic : Circular Queue

```
int dequeue() {
    int data;
    if (front == -1) {
        printf("Queue is Empty");
        return -1;
    }
    data = a[front];
    if (front == rear)
        front = rear = -1;
    else
        front = (front+1) % Max;
    return data;
}
```

front = 4
rear = 2



delete front & rear

front = 0
rear = 2



Slide



Topic : Circular Queue

#Q. Suppose we have a circular queue of size 10 following operation performed

(1) Enqueue(1) Enqueue(2) .... Enqueue(10)

(2) dequeue(), dequeue .... dequeue() 5 times

(3) Enqueue(11) Enqueue(12) .. Enqueue(13) ✓

(4) X = dequeue()

if y is Index of front & z is Index of rear then

$\frac{10x+y-z}{}$ is

$\frac{60+6-2}{} = 64$?

Slide



Topic : Circular Queue

(I) En



front = 0

rear = 9 II

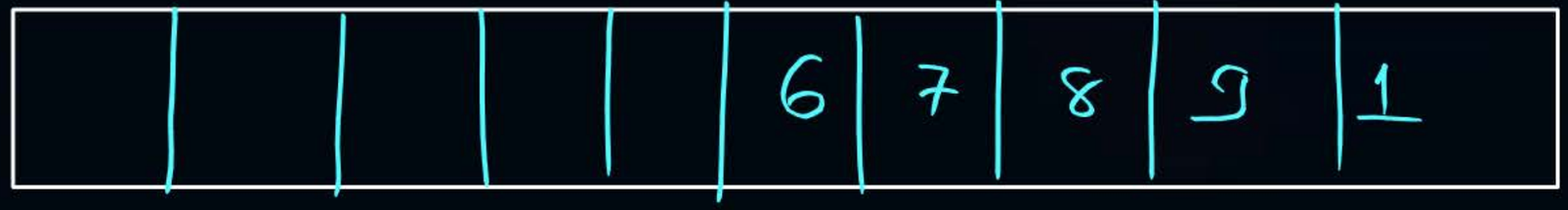


front = 5

rear = 9

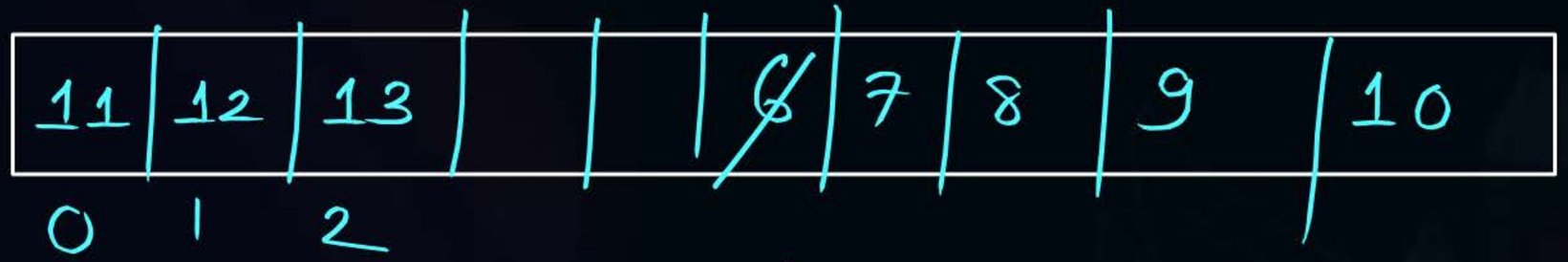
Slide

Topic : Circular Queue



front = 5
rear = 9

front = 5
rear = 2



x = dequeue() = x = 6 , y = 6 z = 2

front = 6

60 + 6 - 2 = 64

Slide



Implementation of Queue Using Stack

push

pop



Stack LIFO

pop will return 4



Queue

Dequeue · will return 1

Slide



Implementation of Queue Using Stack

To take 1 out of stack we Need to temporary store value on top of 1

we use another for holding the value.

Slide

Implementation of Queue Using Stack

Enqueue (1) = push(1)



Auxiliary (Temporary) stack

Stack S1

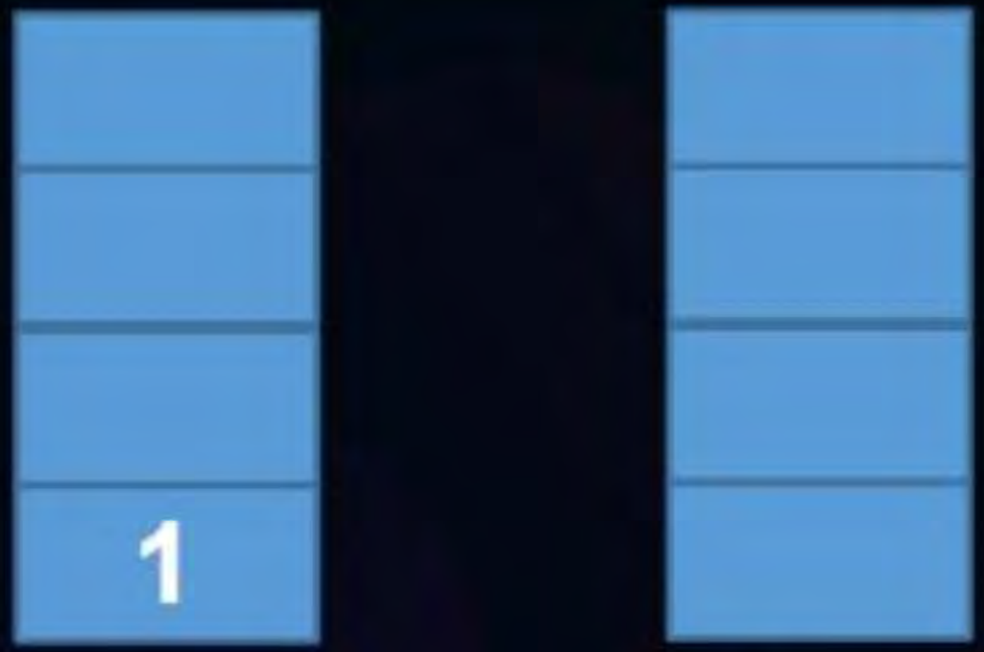
Stack S2

Slide



Implementation of Queue Using Stack

Enqueue(1)

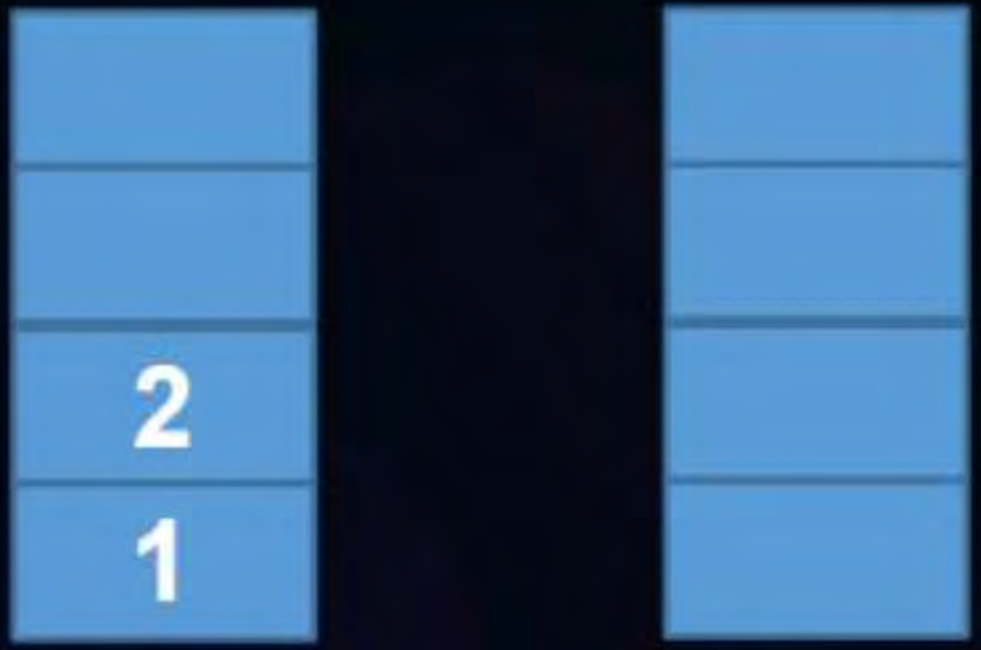


Slide



Implementation of Queue Using Stack

Enqueue(2)



Stack S1

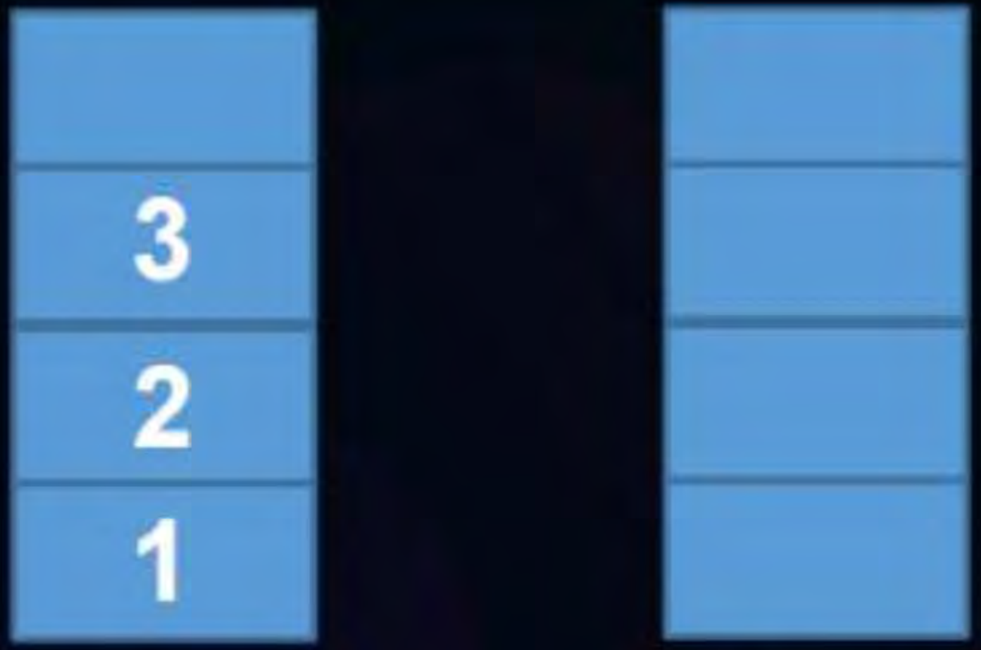
Stack S2

Slide



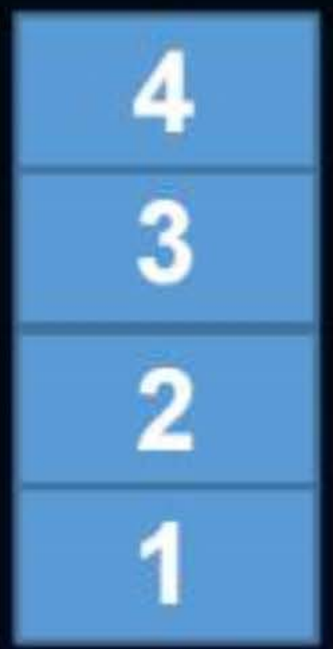
Implementation of Queue Using Stack

Enqueue(3)



Slide

Implementation of Queue Using Stack



Stack S1



Stack S2

Enqueue(4)

4 Enqueue = No. of push 4

n - enqueue = n push

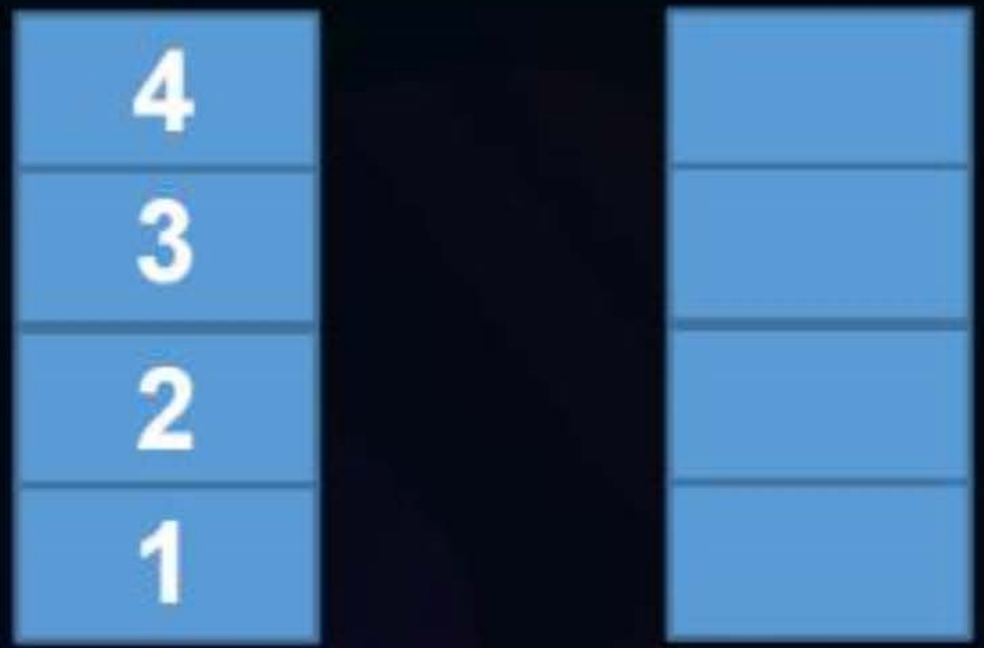
Slide



Implementation of Queue Using Stack



Dequeue



Stack S1

Stack S2

4 pop S_1 } n-pop

4 push S_2 } n-push

Slide



Implementation of Queue Using Stack

Dequeue

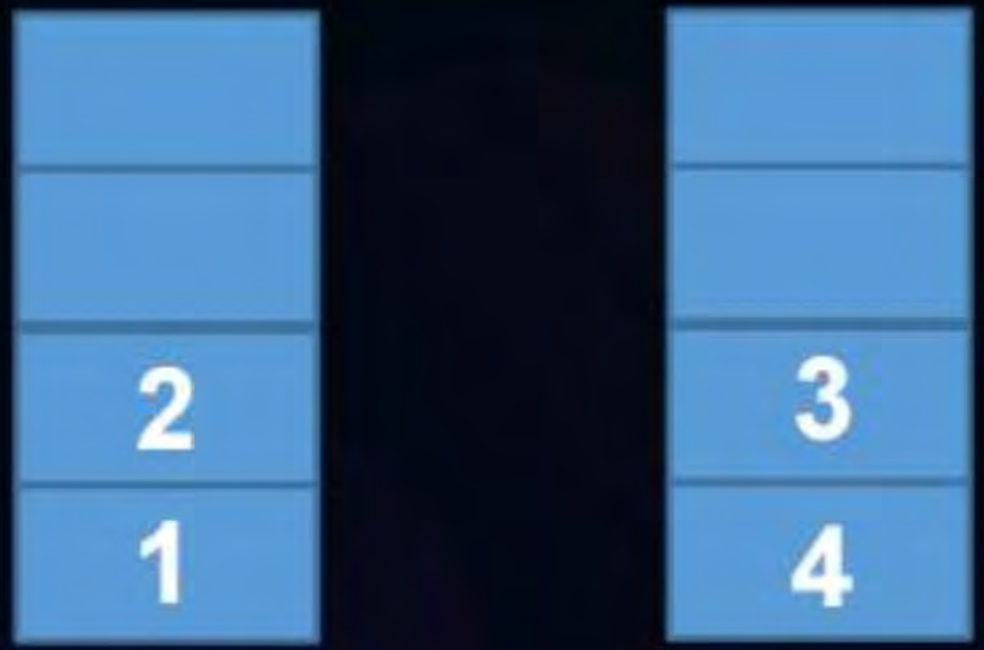


Slide



Implementation of Queue Using Stack

Dequeue

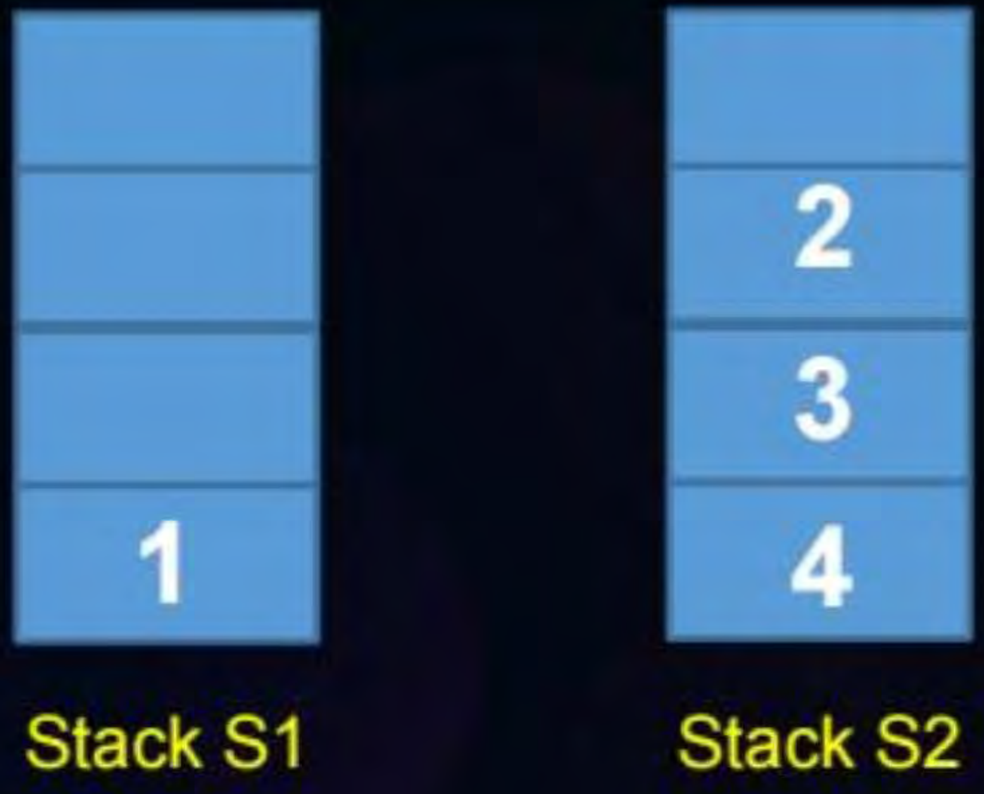


Slide



Implementation of Queue Using Stack

Dequeue

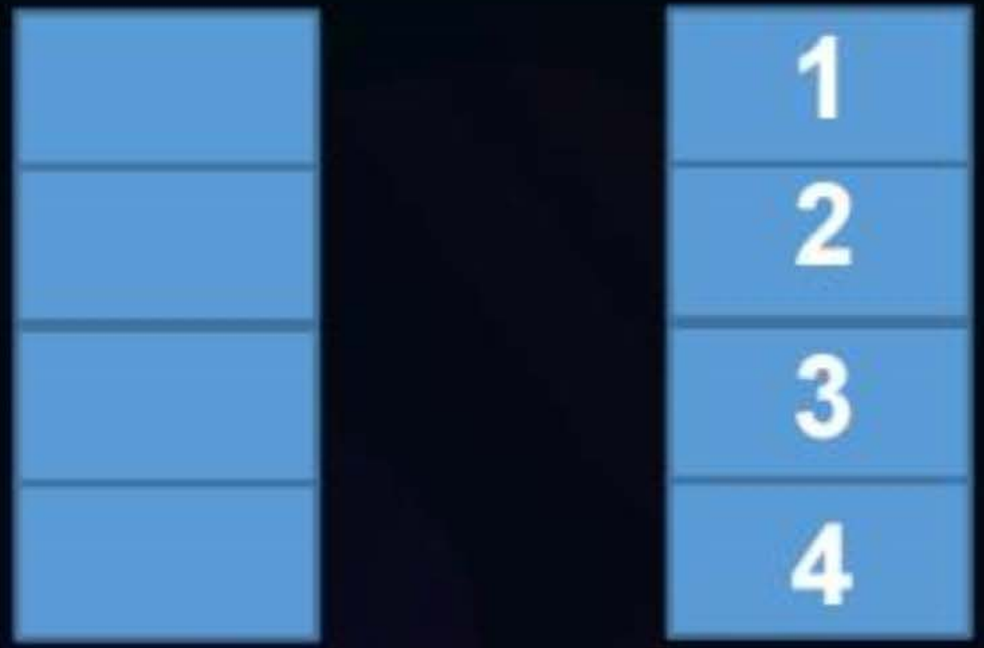


Slide



Implementation of Queue Using Stack

Dequeue



data: pop(S_2) ✓ 1 pop

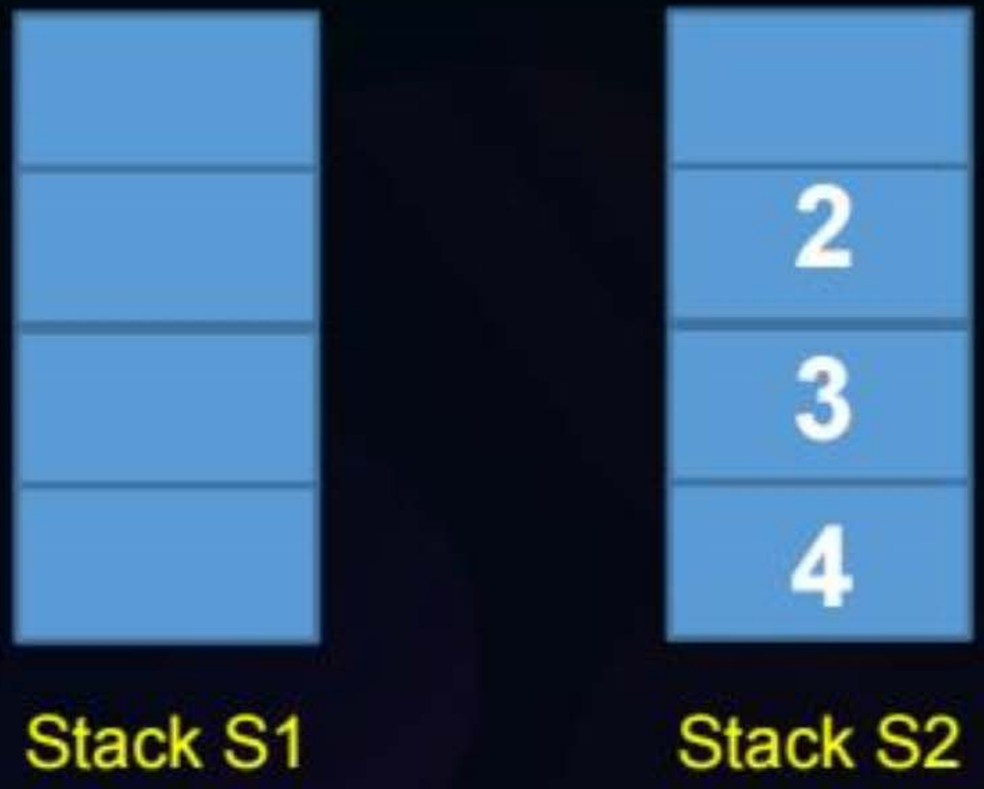
Slide



Implementation of Queue Using Stack



Dequeue



3 pop() S_2

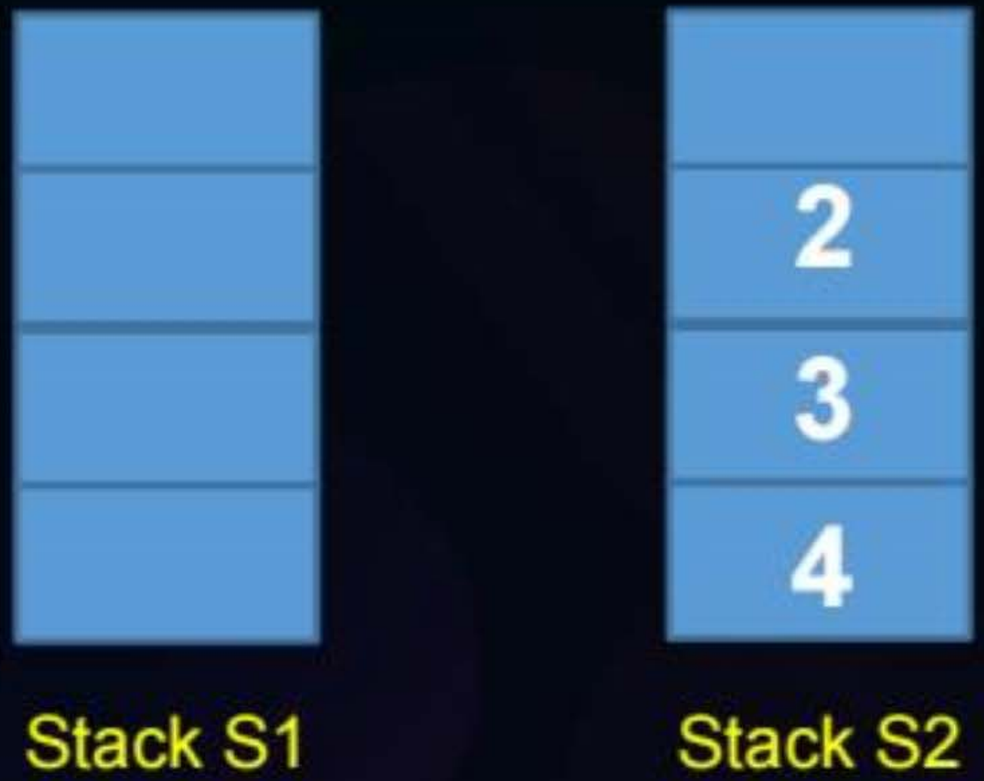
3 push() S_1

Slide



Implementation of Queue Using Stack





Restore

3 pop S_2	n-1 pop
3 push S_1	n-1 push

Slide



Implementation of Queue Using Stack



Restore



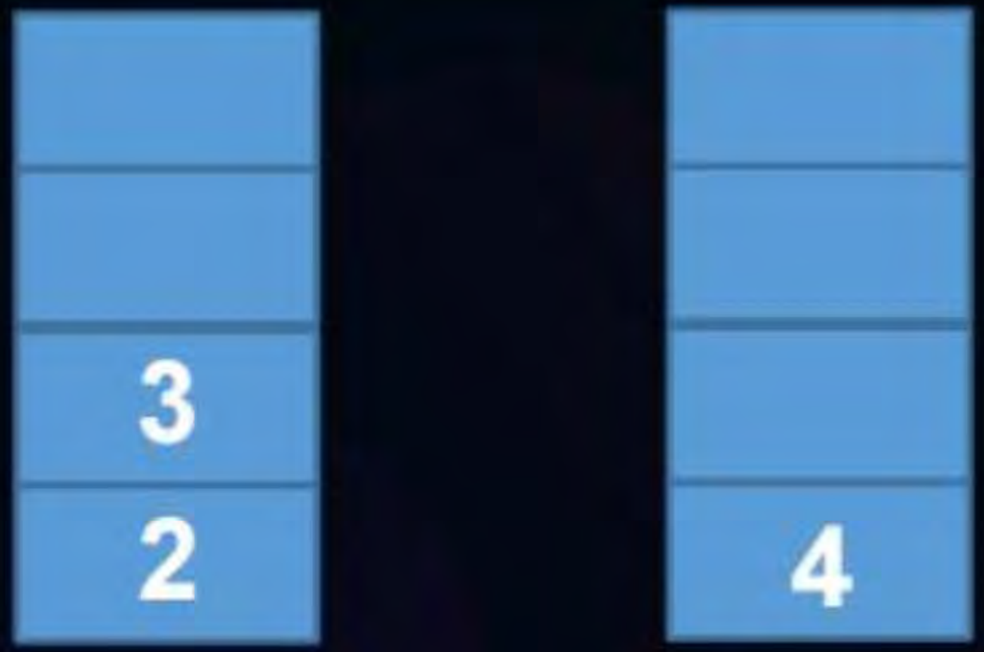
Stack S1

Stack S2



Implementation of Queue Using Stack

Restore



Stack S1

Stack S2

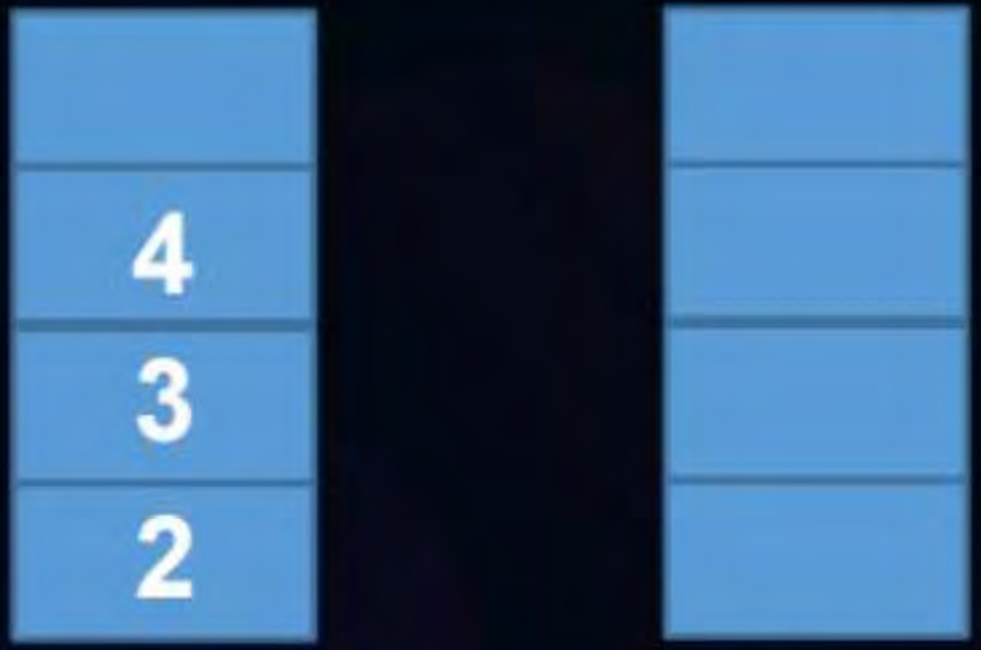
Slide



Implementation of Queue Using Stack



Restore



Stack S1

Stack S2

Slide



Implementation of Queue Using Stack

Enqueue = n push

Dequeue (I) n : pop
n - push

2n pop
2n-1 push

(II) 1 pop()

(III) n-1 pop } Restore
n-1 push }

Slide



Implementation of Queue Using Stack

1 Dequeue

2n pop, 2n-1 push

```
Algorithm Enqueue(x){
    push(S1, x)
}
```

n - Enqueue

n - push

```
Algorithm Dequeue(){
    while(S1 ! empty){
        X = pop(S1);
        push(S2, x);
    }
    data = pop(S2)
    while(S2 ! empty){
        X = pop(S2);
        push(S1, x);
    }
    return data;
}
```

n pop, n push

1 pop

n-1 pop, n-1 push

Slide



Implementation of Queue Using Stack

In the above implementation of queue if 50 Enqueue operation and 2 dequeue operation performed . If total number of push is x and total number pop is y then x+y is

_____

n-Enqueue = n push

1 Dequeue

2n pop

2n-1 push

50 Enqueue - 50 push

1 - Dequeue - 100 pop

99 push

2 Dequeue - 98 pop

97 push

} 49 element

394
50 ←
444

Slide



Implementation of Queue Using Stack

In the above implementation of queue if 50 Enqueue operation and ~~10~~ 4 dequeue operation performed . If total number of push is x and total number pop is y then x+y is

_____

Enqueue : 50 push

dequeue 1 — 100 pop
99

dequeue 2 — 98 pop()
97 push()

dequeue 3- 96 pop
95 push

dequeue 4 - 94 pop
93 push

378
444
1 1
822

Slide

Q. Suppose a stack implementation supports an instruction REVERSE, which reverses the order of elements on the stack, in addition to the PUSH and POP instructions. Which one of the following statements is TRUE with respect to this modified stack?

(GATE_2014_2 M)

time

(A) A queue cannot be implemented using this stack.

(B) A queue can be implemented where ENQUEUE takes a single instruction and DEQUEUE takes a sequence of two instructions.

(C) A queue can be implemented where ENQUEUE takes a sequence of three instructions and DEQUEUE takes a single instruction.

(D) A queue can be implemented where both ENQUEUE and DEQUEUE take a single instruction

Q. Let Q denote a queue containing 3 numbers and S be an empty stack. Head (Q) returns the element at the head of the queue Q without removing it from Q. Similarly Top (S) returns the element at the top of S without removing it from S. Consider the algorithm given below.

```
while Q is not Empty do
    if S is Empty OR Top (S) ≤ Head (Q) then
        X := Dequeue (Q) ;
        Push (S, x) ;
    else
        X : = Pop (S) ;
        Enqueue (Q, X) ;
    end
end
```

The maximum possible number of iterations of the while loop in the algorithm is _____ **(GATE_2016_2 M)**



2 mins Summary

Topic — Circular queue

Topic — stack implementation of queue

Topic — deque : doubly ended queue

Topic — priority queue.

Topic


Linked List
THANK - YOU
Slide